

Лекція 8. Автоматичне тестування

Автоматичне тестування – це процес перевірки програмного забезпечення з використанням спеціальних інструментів та скриптів для виконання тестів без втручання людини. Воно є невід’ємною частиною сучасної розробки програмного забезпечення та спрямоване на оптимізацію процесу тестування, підвищення ефективності та мінімізацію людського фактора. Завдяки автоматизації тестування можна швидше знаходити помилки, забезпечувати стабільність коду та спрощувати інтеграцію нових функціональностей. Це дозволяє зменшити час виходу продукту на ринок, скоротити витрати на підтримку, а також підвищити загальну якість програмного забезпечення.

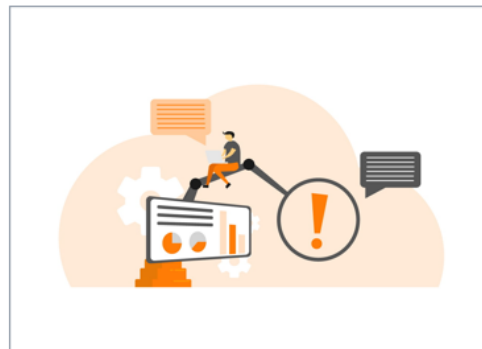
Що таке автоматичне тестування?

Це процес використання спеціальних програм або інструментів для автоматичної перевірки якості програмного забезпечення.

Замість ручного тестування скрипти самостійно перевіряють функціональність, безпеку, продуктивність додатку.

Мета лекції:

- Ознайомити з основами автоматичного тестування.
- Пояснити різні типи автоматичних тестів.
- Показати роль тестування у процесах CI/CD.



Основи автоматичного тестування

Автоматичне тестування – це процес використання спеціалізованих інструментів і скриптів для автоматизації виконання тестових сценаріїв, які перевіряють різні аспекти програмного забезпечення. Воно дозволяє проводити регулярні, повторювані та масштабовані перевірки, зменшуючи потребу в ручному втручанні.

Автоматизація виконує завдання, які займають багато часу під час ручного тестування, наприклад, повторювані перевірки або тестування великих наборів даних. Проте, у деяких випадках ручне тестування залишається необхідним, наприклад, для оцінки UX або перевірки нестандартних сценаріїв.

Основи автоматичного тестування



Переваги:

- Швидкість перевірки змін.
- Зниження людського фактора.
- Повторюваність тестів.



Недоліки:

- Витрати часу на створення тестів.
- Підтримка актуальності тестових скриптів.



- Мета: Раннє виявлення помилок у коді.

Тестові скрипти – це автоматизовані сценарії, які виконують перевірки функціональності, продуктивності, безпеки, сумісності тощо. Ці скрипти можуть бути написані мовами програмування (наприклад, Python, Java) або створені у спеціалізованих середовищах для автоматичного тестування (наприклад, Selenium або Cypress).

Завдяки автоматизації тестування можна швидко виявляти помилки та забезпечувати високу якість програмного забезпечення. Це особливо важливо в умовах сучасного Agile-розроблення та CI/CD процесів, де релізи відбуваються часто й швидко.

Автоматизація дозволяє розробникам одразу дізнаватися про помилки в нових змінах завдяки швидкому запуску тестів. Тести можна запускати багаторазово в однакових умовах, що забезпечує стабільність і передбачуваність результатів. Автоматичне тестування легко адаптується до великих проєктів, де тестування вручну може бути занадто складним або тривалим. Незважаючи на початкові витрати на розробку тестів, автоматизація значно зменшує витрати на тестування під час тривалого циклу життя проєкту.

Навіщо потрібне автоматичне тестування?

Перевірка стабільності при великих обсягах змін.

Підвищення надійності продукту перед релізом.

Автоматизація регресійного тестування.

Швидке виявлення помилок під час розробки.



Типи завдань, які вирішуються за допомогою автоматизації:

- перевірка правильної роботи базового функціоналу (юніт-тести);
- автоматизація перевірок під час інтеграції кількох модулів (інтеграційні тести);
- тестування інтерфейсу користувача (UI тести);
- перевірка продуктивності під час пікових навантажень (продуктивні тести);
- тестування безпеки, щоб уникнути уразливостей у системі.

Переваги автоматичного тестування

Швидкість виконання тестів

- автоматизовані тести виконуються набагато швидше, ніж ручні, оскільки вони не залежать від людського фактора. Це особливо важливо для великих проєктів, де кількість тестів може обчислюватися сотнями або тисячами;
- завдяки швидкості автоматизації тестування розробники можуть отримувати результати одразу після внесення змін у код, що значно прискорює цикл розробки.

Можливість повторного використання тестів

- один раз створені автоматизовані тести можуть використовуватися багаторазово, що робить їх ефективними для регресійного тестування;
- тести можна налаштувати для виконання на різних платформах, пристроях або середовищах, що забезпечує гнучкість і масштабованість;
- При оновленні програмного забезпечення можна легко адаптувати існуючі тести, що економить час і ресурси.

Раннє виявлення помилок

- Автоматичне тестування дозволяє швидко виявляти помилки на ранніх етапах розробки, зменшуючи витрати на їх виправлення;
- Інтеграція автоматичних тестів у процеси CI/CD забезпечує постійну перевірку кожного нового коміту, що дозволяє уникати накопичення дефектів у коді;
- Це зменшує ризики, пов'язані з релізом, і забезпечує високу якість продукту.

Підвищення стабільності релізів:

- постійне автоматичне тестування допомагає уникнути ситуацій, коли неочікувані помилки потрапляють у продуктивне середовище;
- завдяки ретельному тестуванню всієї системи забезпечується стабільність релізів, навіть якщо продукт активно розвивається;
- регресійне тестування дозволяє перевірити, що нові зміни не порушують існуючу функціональність, що робить релізи більш передбачуваними.

Додаткові переваги:

- *зниження навантаження на команду тестувальників:* автоматизація зменшує обсяг рутинної роботи, дозволяючи тестувальникам зосередитися на складніших і творчих завданнях, наприклад, тестуванні користувацького досвіду або нестандартних сценаріїв;

- *підвищення точності*: виконання тестів автоматичними інструментами усуває ризики, пов'язані з людськими помилками під час ручного тестування;
- *економія часу та ресурсів*: хоча розробка автоматичних тестів вимагає початкових інвестицій, у довгостроковій перспективі це знижує витрати на підтримку та тестування;
- *масштабованість*: автоматизовані тести легко адаптуються до великих проєктів або систем із багатокомпонентною архітектурою.

Інструменти для автоматичного тестування

Сучасні інструменти автоматичного тестування пропонують широкий спектр можливостей для перевірки функціональності, продуктивності, безпеки, а також інтеграції та розгортання. Нижче наведено деякі з найбільш популярних інструментів, їх призначення та особливості.

Популярні інструменти для автоматичного тестування

- **JUnit** — для тестування Java-додатків.
- **Selenium** — для автоматизації браузерного тестування.
- **Pytest** — для тестування на Python.
- **Postman/Newman** — для тестування API.
- **GitHub Actions / Jenkins / GitLab CI** — для інтеграції тестів у CI/CD процеси.



Selenium – це відкрита платформа для автоматизації тестування веб-додатків, яка широко використовується завдяки своїй гнучкості та підтримці різних мов програмування, таких як Java, Python, C#, Ruby, JavaScript. Інструмент дозволяє тестувати веб-додатки на різних браузерах, включаючи Chrome, Firefox, Edge та Safari, що забезпечує кросбраузерну сумісність. Для

розподіленого тестування використовується Selenium Grid, що дає можливість виконувати тести паралельно на різних машинах. Selenium найчастіше застосовується для перевірки інтерфейсу користувача (UI), забезпечуючи автоматизацію взаємодії з веб-сторінками.

JUnit є інструментом для автоматизації юніт-тестів у Java. Він дозволяє ефективно писати, організовувати та запускати юніт-тести. Серед його можливостей – інтеграція з багатьма інструментами CI/CD, такими як Jenkins, а також підтримка анотацій, що спрощують процес написання тестів. JUnit є одним із найпоширеніших фреймворків серед Java-розробників.

TestNG є розширеним фреймворком для автоматичного тестування Java-додатків. Він забезпечує підтримку паралельного запуску тестів, інтеграцію з інструментами звітності та гнучку структуру, яка дозволяє виконувати тести залежно від їх пріоритету. Цей фреймворк часто використовується разом із Selenium для розширення можливостей автоматизації.

PyTest – це фреймворк для тестування Python-додатків, що відзначається простотою використання завдяки зрозумілому синтаксису. Він дозволяє створювати як юніт-, так і інтеграційні тести, а також підтримує плагіни для розширення функціональності. PyTest особливо зручний для автоматизації тестування веб-додатків і API.

Cypress – це сучасний інструмент для тестування веб-додатків, який особливо популярний у фронтенд-розробці. Він забезпечує просте налаштування та швидку інтеграцію з проєктом, підтримує інтерактивне тестування в реальному часі та дозволяє тестувати як інтерфейс користувача, так і API. Cypress чудово підходить для роботи з сучасними JavaScript/TypeScript фреймворками, такими як React, Angular і Vue.js.

Jenkins – це інструмент автоматизації, широко використовуваний у процесах CI/CD. Він дозволяє автоматизувати запуск тестів після кожного коміту, інтегрується з більшістю інструментів тестування та розгортання, а також підтримує плагіни для розширення функціональності. Jenkins часто

застосовується для створення конвеєрів, які включають автоматичне тестування, збірку та деплой додатків.

Apache JMeter – це інструмент для тестування продуктивності та навантаження, який дозволяє перевіряти продуктивність веб-додатків, баз даних і серверів. Він підтримує багатопотокове тестування та створення сценаріїв із великими обсягами даних. JMeter широко використовується для тестування систем під високим навантаженням.

Postman – це інструмент для тестування API, який забезпечує автоматизацію тестування RESTful API, підтримує колекції запитів і створення автоматичних тестових сценаріїв. Він інтегрується з CI/CD, дозволяючи перевіряти API на кожному етапі розробки. Postman є простим у використанні та підходить як для початківців, так і для професіоналів.

Robot Framework – це гнучкий інструмент для автоматичного тестування, заснований на ключових словах. Він підтримує різні типи тестів, зокрема юніт-, інтеграційні та регресійні, і може інтегруватися з бібліотеками Python. Завдяки використанню ключових слів забезпечується створення читабельних тестів. Robot Framework зручний для великих команд, учасники яких мають різний рівень досвіду.

Allure – це інструмент для створення звітів про виконання автоматичних тестів. Він генерує зрозумілі та привабливі звіти, інтегрується з більшістю інструментів тестування, таких як JUnit, TestNG, PyTest, і відображає результати тестування у вигляді діаграм і графіків. Allure забезпечує зручну візуалізацію результатів, що сприяє аналітиці та прийняттю рішень.

Типи автоматичних тестів

Юніт-тести (Unit Testing):

- Перевірка окремих функцій або методів.

Інтеграційні тести (Integration Testing):

- Перевірка взаємодії між модулями.

Системні тести (System Testing):

- Тестування всього додатку як цілісної системи.

Тести приймання (Acceptance Testing):

- Перевірка відповідності вимогам замовника.

Тести безпеки:

- Перевірка вразливостей системи.

Тести навантаження (Load Testing):

- Визначення, як система працює під високим навантаженням.

Тести зручності (UI Testing):

- Автоматизація перевірки інтерфейсу користувача.

Юніт-тести (Unit Tests)

Юніт-тести перевіряють роботу окремих модулів або функцій програми в ізольованому середовищі. Вони спрямовані на виявлення помилок у конкретних частинах коду та забезпечення їхньої коректної роботи.

Приклад використання: якщо є функція для підрахунку суми двох чисел, юніт-тест перевірить, чи повертається правильний результат для різних комбінацій введених значень.

```
python

def test_addition():
    assert add(2, 3) == 5
```

Переваги юніт-тестів: легко виявляти помилки на ранніх етапах розробки. Швидке виконання забезпечується тестуванням невеликих частин коду. Також спрощується рефакторинг, адже тести підтверджують, що зміни не порушили основний функціонал.

Серед інструментів для юніт-тестування можна виділити кілька популярних рішень. JUnit є найпоширенішим інструментом для написання та виконання тестів у Java-додатках. NUnit використовується для юніт-тестування у середовищі .NET. PyTest є універсальним фреймворком для написання тестів у Python, який також підтримує юніт-тестування. Для тестування коду на C++ часто використовують Google Test, який є популярним і ефективним інструментом.

Особливості юніт-тестів:

- виконуються ізольовано, без взаємодії з базами даних, API або зовнішніми сервісами;
- використовуються макети (mocking) для заміщення залежностей та перевірки тільки того коду, що тестується.

Приклад тестування: якщо є функція, яка визначає, чи є число парним:

```
python

def is_even(num):
    return num % 2 == 0

def test_is_even():
    assert is_even(4) == True
    assert is_even(5) == False
```

Хоча юніт-тести є базовим типом тестування, вони часто поєднуються з іншими видами для забезпечення комплексного підходу. Інтеграційні тести перевіряють взаємодію між різними модулями. Системні тести оцінюють функціонування всієї системи. Регресійні тести допомагають упевнитися, що нові зміни не порушили існуючий функціонал. UI- або End-to-End тести охоплюють перевірку повного сценарію взаємодії користувача із системою.

Інтеграційні тести (Integration Tests)

Інтеграційні тести перевіряють взаємодію між окремими модулями або компонентами системи, щоб переконатися, що вони працюють разом правильно. Цей тип тестування є важливим для виявлення проблем, які можуть виникнути на стиках між модулями.

Інтеграційні тести перевіряють, як один модуль передає дані іншому, чи правильно виконується логіка взаємодії між компонентами, а також сумісність компонентів, які залежать один від одного.

Приклади використання:

1) *Веб-додаток*: перевірка взаємодії між фронтом (React, Angular, Vue.js) і бекендом (Node.js, Django, Spring). Наприклад, тест перевіряє, чи API повертає правильні дані, а фронтенд відображає їх коректно.

```
python

def test_api_integration():
    response = requests.get("https://example.com/api/data")
    assert response.status_code == 200
    assert "name" in response.json()
```

2) *мобільний додаток*: перевірка роботи між мобільним клієнтом і сервером через REST або GraphQL API.

3) *бази даних*: тестування взаємодії між бекендом і базою даних, наприклад, перевірка, чи правильно записуються або читаються дані.

Основні підходи до інтеграційного тестування:

- **Big Bang Testing**: усі модулі інтегруються одночасно, і система тестується як є;
- **Incremental Testing**: модулі інтегруються поступово й тестуються на кожному кроці;

- **Top-Down Testing:** тестування починається з високорівневих модулів і поступово переходить до нижчого рівня;
- **Bottom-Up Testing:** спочатку тестуються модулі нижнього рівня, а потім додаються модулі верхнього рівня.

Переваги інтеграційних тестів полягають у можливості виявляти помилки в логіці взаємодії модулів. Вони забезпечують перевірку реальної поведінки системи, а не лише її окремих частин, а також дозволяють переконатися, що система відповідає вимогам замовника.

Розглянемо веб-додаток, де користувач заповнює форму на фронтенді, а бекенд обробляє дані й записує їх у базу даних. Інтеграційний тест може виглядати так:

```
def test_form_submission_integration():  
    # Імітація заповнення форми на фронтенді  
    form_data = {"username": "test_user", "email": "test@example.com"}  
    response = requests.post("https://example.com/api/submit", json=form_data)  
  
    # Перевірка відповіді сервера  
    assert response.status_code == 201  
  
    # Перевірка, чи дані записані в базу  
    db_response = db.query("SELECT * FROM users WHERE email = 'test@example.com'")  
    assert db_response["username"] == "test_user"
```

Серед інструментів для інтеграційного тестування можна виділити кілька ефективних рішень. Postman використовується для тестування API, перевірки запитів та їх взаємодії з бекендом. SoapUI підходить для тестування SOAP і REST API, особливо для складних інтеграцій. Cucumber дозволяє писати тести у вигляді зрозумілих сценаріїв, що часто використовується в підході BDD. Spring Test є фреймворком для інтеграційного тестування Java-додатків із використанням Spring. Комбінація JUnit із Mock Server підходить для перевірки інтеграції з базами даних і зовнішніми API у Java-проектах. У Python для

написання інтеграційних тестів можна використовувати PyTest разом із бібліотеками mock або requests.

Інтеграційні тести часто потребують налаштування тестового середовища, яке імітує реальні умови роботи системи. Це може включати використання локальних серверів, макетів API або тестових баз даних. Для складних систем із великою кількістю компонентів такі тести дозволяють виявляти проблеми, які важко передбачити на рівні юніт-тестування.

Функціональні тести (Functional Tests)

Функціональні тести перевіряють, чи працює програмне забезпечення відповідно до специфікацій і вимог. Основна мета – перевірити функціональність системи з точки зору користувача, забезпечуючи правильну роботу окремих функцій і сценаріїв.

Функціональні тести зосереджені на перевірці того, що система виконує, а не на тому, як саме це реалізовано. Вони здійснюються за методом тестування чорного ящика, без необхідності глибокого розуміння внутрішньої реалізації коду. Основою таких тестів зазвичай є сценарії, що імітують реальну поведінку користувача.

Приклади функціональних тестів:

1) **Форма реєстрації користувача.** Користувач може зареєструватися, ввівши коректні дані (ім'я, email, пароль).

Сценарії тестування:

- заповнення всіх полів правильно – реєстрація має бути успішною;
- введення некоректного email – система має показати відповідне повідомлення про помилку;
- введення вже існуючого email – користувач отримує сповіщення, що такий email вже зареєстрований.

Приклад коду тесту (Python):

```
def test_user_registration():
```

```
    data = {"username": "test_user", "email": "test@example.com", "password": "password123"}
```

```
response = requests.post("https://example.com/api/register", json=data)
assert response.status_code == 201
assert response.json()["message"] == "User registered successfully."
```

2) **Авторизація.** Перевіряють: введення коректного логіна і пароля дозволяє отримати доступ. Неправильний пароль або логін – показується повідомлення про помилку.

3) **Додавання товару до кошика.** Додавання товару до кошика перевіряє кілька важливих аспектів. Перш за все, чи товар додається до кошика і чи його кількість відображається коректно. Також тестується правильність обчислення загальної суми. Окрім цього, перевіряється, що видалення товару з кошика працює без помилок.

4) **Обробка платежів.** Обробка платежів включає перевірку коректності введення платіжних даних і тестування завершення транзакції. Сценарії охоплюють як успішне, так і неуспішне виконання платіжних операцій.

Переваги функціональних тестів: вони забезпечують перевірку виконання всіх заявлених функцій системи. Такі тести легко зрозумілі замовникам, оскільки фокусуються на перевірці конкретних сценаріїв. Вони також допомагають переконатися, що продукт відповідає бізнес-вимогам.

До недоліків функціональних тестів належить те, що вони охоплюють лише зовнішню функціональність, залишаючи без уваги внутрішню реалізацію. Крім того, для великих проєктів такі тести можуть бути часозатратними, якщо не використовується автоматизація.

Інструменти для функціонального тестування включають Selenium, який застосовується для автоматизації тестів веб-додатків та створення скриптів для перевірки інтерфейсу користувача. Cypress підходить для роботи з сучасними JavaScript-фреймворками та створення інтерактивних тестів. TestComplete є універсальним інструментом для тестування веб, мобільних і десктопних додатків. Postman використовується для перевірки функціональності REST API,

а QTP/UFT від Micro Focus є популярним рішенням для тестування додатків у різних середовищах.

Тести інтерфейсу користувача (UI Tests)

Тести інтерфейсу користувача (UI Tests) спрямовані на перевірку вигляду, взаємодії та функціональності елементів користувацького інтерфейсу. Їх основна мета – переконатися, що програма виглядає та працює відповідно до очікувань користувача.

Такі тести оцінюють коректність відображення елементів інтерфейсу, як-от кнопки, поля вводу чи таблиці, та перевіряють їхню реакцію на дії користувача, зокрема натискання, введення тексту чи навігацію. Вони також забезпечують відповідність дизайну до специфікацій, включаючи кольори, шрифти та розташування елементів.

Окрім цього, увага приділяється сумісності інтерфейсу з різними роздільними здатностями екранів, перевірці адаптивності та інтерактивних елементів, таких як випадаючі меню, чекбокси чи модальні вікна. Важливою частиною тестів є перевірка зворотного зв'язку, наприклад, відображення повідомлень про помилки при некоректному введенні даних.

Приклади використання:

1) **Форма входу**: перевірка, чи відображається поле для логіна та пароля. Тестування коректності кнопки «Увійти»: вона має стати активною лише після заповнення полів. Перевірка, чи з'являється відповідне повідомлення, якщо логін або пароль некоректні.

3) **Кошик у веб-магазині**: перевірка, чи можна додати товар до кошика. Перевірка, чи правильно оновлюється кількість товарів. Тестування кнопок «Оформити замовлення» та «Видалити товар».

4) **Мобільний додаток**: перевірка коректного відображення елементів на різних пристроях (телефони, планшети). Тестування жестів: свайпи, довгі натискання.

Інструменти для тестування інтерфейсу включають кілька популярних рішень. Selenium є одним із найпоширеніших фреймворків для автоматизації тестування веб-додатків. Він підтримує різні мови програмування, такі як Python, Java, та C#, і дозволяє автоматизувати взаємодію з елементами інтерфейсу, наприклад, натискання кнопок, введення тексту або навігацію між сторінками.

Cypress – сучасний інструмент для автоматизації тестів веб-додатків. Він простий у налаштуванні, працює безпосередньо в браузері та підтримує інтеграцію з CI/CD платформами, такими як Jenkins чи GitHub Actions.

TestComplete є універсальним інструментом для UI-тестування десктопних, веб і мобільних додатків. Він дозволяє створювати тести як за допомогою запису дій користувача, так і через написання скриптів.

Appium спеціалізується на тестуванні інтерфейсу мобільних додатків для платформ Android та iOS. Він підтримує автоматизацію жестів та роботу з різними елементами мобільного інтерфейсу.

Playwright – новий інструмент для автоматизації браузерів, який підходить для роботи з сучасними веб-додатками, що активно використовують JavaScript. Він підтримує такі мови програмування, як JavaScript, Python і C#.

Переваги тестів інтерфейсу полягають у тому, що вони забезпечують перевірку системи з точки зору кінцевого користувача. Такі тести дозволяють виявляти проблеми, які можуть залишатися непоміченими на рівні юніт- або інтеграційних тестів, і охоплюють увесь користувацький сценарій, наприклад, від авторизації до оформлення замовлення.

Проте вони мають і певні недоліки. Тести інтерфейсу можуть бути досить часозатратними, особливо для великих проєктів. Вони потребують регулярного оновлення у разі змін у дизайні або логіці інтерфейсу, а також є складними для підтримки, якщо інтерфейс часто змінюється.

Приклад UI тесту: Перевірка кнопки «Додати в кошик»:

```
python

from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://example.com/product-page")

# Натискання кнопки "Додати в кошик"
add_to_cart_button = driver.find_element("id", "add-to-cart")
add_to_cart_button.click()

# Перевірка, чи товар додано до кошика
cart = driver.find_element("id", "cart-items")
assert "Product Name" in cart.text
driver.quit()
```

Продуктивні тести (Performance Tests)

Продуктивні тести призначені для оцінки ключових характеристик системи, таких як швидкодія, стабільність, масштабованість і надійність під різними навантаженнями. Вони допомагають визначити, чи відповідає програмне забезпечення вимогам до продуктивності.

Такі тести перевіряють швидкодію, зокрема час завантаження сторінок, відповіді сервера та виконання запитів до бази даних. Вони оцінюють масштабованість, визначаючи, як система реагує на збільшення кількості користувачів або обсягу даних. Тести також перевіряють стабільність, забезпечуючи виявлення збоїв при тривалих навантаженнях. Крім того, оцінюється пропускна здатність, тобто кількість запитів або операцій, які система здатна обробити за одиницю часу.

Типи продуктивних тестів:

1) тести навантаження (Load Testing):

- визначають, як система працює під нормальним і піковим навантаженням;
- приклад: 500 одночасних користувачів заходять на вебсайт і завантажують сторінку продукту.

2) стрес-тестування (Stress Testing):

- визначає, як система поводить ся за межами своїх можливостей (коли ресурси перевантажені);
- приклад: спроба обробити 10 000 одночасних запитів, коли сервер розрахований на 5 000.

3) тестування стабільності (Stability/Soak Testing):

- перевіряє, чи може система працювати тривалий час під постійним навантаженням без деградації продуктивності;
- приклад: сервер обробляє 1 000 запитів щогодини протягом 48 годин.
- 4) тестування пропускнуої здатності (Capacity Testing): визначає максимальну кількість користувачів або операцій, які система може обробити без деградації продуктивності.

Приклади продуктивних тестів:

1) *Тестування веб-додатка.*

Мета: Виміряти час завантаження головної сторінки для 1 000 користувачів, які одночасно відвідують сайт.

Інструмент: JMeter

```
bash  
  
jmeter -n -t test-plan.jmx -l results.jtl
```

Результати аналізують у вигляді графіків часу відповіді для кожного запиту.

2) *Тестування бази даних.*

Мета: Перевірити, чи здатна база даних обробляти 10 000 запитів на секунду.

Інструмент: Gatling

```
scala

val scn = scenario("Database Load Test")
  .exec(http("Query DB")
    .get("/database/query"))
  setUp(scn.inject(atOnceUsers(10000))).protocols(httpProtocol)
```

Інструменти для продуктивного тестування включають кілька ефективних рішень. JMeter є одним із найпопулярніших інструментів для навантажувального тестування, підтримуючи протоколи HTTP, FTP, JDBC, SOAP, REST та інші. Він дозволяє створювати складні сценарії навантаження.

Gatling – потужний інструмент на основі мови Scala, який підходить для інтеграції в CI/CD процеси. Він забезпечує високу масштабованість і детальне логування результатів.

k6 – це простий у використанні інструмент для тестування продуктивності API, написаний на JavaScript, що робить його зручним для веб-розробників.

LoadRunner є корпоративним рішенням для стресового та навантажувального тестування з широкими можливостями аналізу та звітності.

Apache Bench (ab) пропонує легке рішення для швидкого тестування HTTP-запитів, а Locust, створений на основі Python, підтримує розробку сценаріїв користувацьких потоків для ефективного тестування навантаження.

Переваги продуктивних тестів полягають у тому, що вони допомагають виявити вузькі місця системи ще до її реального використання, підвищуючи надійність і стабільність продукту. Такі тести дозволяють прогнозувати поведінку системи за різних сценаріїв, забезпечуючи краще розуміння її можливостей.

Разом із тим, продуктивні тести мають і недоліки. Вони потребують значних ресурсів для моделювання реального навантаження, вимагають ретельного налаштування і точного вимірювання параметрів. Крім того, результати тестів можуть бути складними для інтерпретації.

Регресійні тести (Regression Tests)

Регресійні тести призначені для перевірки того, чи впровадження нових змін або функціональностей не порушило вже існуючий функціонал. Вони є одним із ключових елементів забезпечення якості, особливо в проєктах із частими оновленнями, де ризик виникнення побічних ефектів досить високий.

Мета таких тестів – гарантувати, що програмний продукт працює так само добре, як і раніше, навіть після внесення модифікацій у код. Це дозволяє зберігати стабільність системи та уникати неочікуваних помилок у функціональності.

Приклади використання:

- після виправлення помилки проводяться регресійні тести, щоб переконатися, що це виправлення не спричинило проблем в інших частинах програми;
- перед випуском нової версії програми регресійні тести допомагають перевірити, чи всі основні функції працюють належним чином.

Інструменти для регресійного тестування включають кілька ефективних рішень. Selenium забезпечує автоматизоване тестування веб-додатків і використовується для перевірки регресії. TestComplete є універсальним інструментом для автоматизації як функціональних, так і регресійних тестів. PyTest спеціалізується на регресійному тестуванні Python-додатків. Jenkins дозволяє інтегрувати регресійні тести в процеси CI/CD, забезпечуючи постійний моніторинг стабільності системи.

Регресійні тести можуть бути повними (всі частини системи перевіряються) або вибірковими (тестуються лише частини, які могли постраждати від змін). Їх автоматизація є надзвичайно важливою в сучасних підходах до розробки, таких як Agile або DevOps, оскільки вона знижує витрати часу та людських ресурсів, забезпечуючи водночас якість та стабільність.

Тести безпеки (Security Tests)

Тести безпеки є важливим інструментом для забезпечення надійності та захисту програмного забезпечення від потенційних загроз, атак чи зловживань. Їх головна мета – виявлення вразливостей, перевірка рівня безпеки та забезпечення відповідності додатка сучасним стандартам кібербезпеки.

Такі тести допомагають ідентифікувати можливі точки входу для атак, зокрема SQL-ін'єкції, XSS чи CSRF. Вони перевіряють коректність реалізації механізмів автентифікації та авторизації, оцінюють рівень захищеності конфіденційних даних, таких як паролі, фінансова інформація або персональні дані користувачів. Крім того, тести безпеки забезпечують перевірку правильного використання шифрування та інших протоколів безпеки, а також визначають здатність системи протистояти атакам типу DoS/DDoS.

Приклади тестування:

- перевірка, чи можуть неавторизовані користувачі отримати доступ до захищених ресурсів;
- моделювання атак, наприклад, SQL-ін'єкцій або міжсайтового виконання скриптів (XSS);
- оцінка того, як система поводить себе в умовах підвищеного навантаження або при спробах злоумисників зламати шифрування даних.

Інструменти для тестування безпеки:

- OWASP ZAP (Zed Attack Proxy) – це інструмент з відкритим кодом для пошуку вразливостей у веб-додатках. Він дозволяє виконувати сканування, аналізувати безпеку та виявляти слабкі місця.
- Burp Suite – потужний комплекс інструментів для тестування безпеки, який підтримує моделювання атак, перевірку вразливостей і тестування взаємодії між клієнтом та сервером.
- Nessus – інструмент для сканування мережі та виявлення вразливостей у конфігураціях.
- Metasploit – платформа для моделювання атак та перевірки захисту системи.

- SQLMap – автоматизований інструмент для виявлення SQL-ін'єкцій.
- Arachni – інструмент для тестування безпеки веб-додатків, який виявляє загальні типи вразливостей.

Переваги тестів безпеки включають забезпечення захищеності системи від зловмисників, виявлення потенційних вразливостей до їх експлуатації, дотримання стандартів і норм безпеки, таких як GDPR або PCI DSS. Крім того, вони сприяють зниженню репутаційних і фінансових ризиків, пов'язаних із витоком даних або кібератаками.

Регулярне тестування безпеки особливо важливе для фінансових, медичних і державних систем, де захист даних має критичне значення. Автоматизація тестів безпеки у поєднанні з регулярним ручним аудитом допомагає забезпечити найвищий рівень захисту додатків.

Автоматичні тести в CI/CD

Автоматичне тестування є ключовим елементом у процесах CI/CD, оскільки забезпечує ефективність і надійність розробки програмного забезпечення. Його використання дозволяє командам швидко і безпечно впроваджувати зміни, зберігаючи високу якість коду.

Стабільність коду є одним із основних досягнень завдяки автоматичному тестуванню. Тести дозволяють оперативно виявляти дефекти та помилки до того, як код потрапить у продуктивне середовище. Це знижує ризик збоїв системи, підвищує довіру користувачів і зменшує витрати на виправлення помилок на пізніх етапах.

Швидка перевірка змін допомагає розробникам зосередитися на створенні нової функціональності, не витрачаючи багато часу на ручне тестування. Автоматизація дозволяє виконувати сотні або навіть тисячі тестів одночасно, прискорюючи процес валідації функціоналу. Завдяки цьому кожна зміна може бути перевірена відразу після її внесення.

Запобігання помилкам на ранніх етапах розробки є критичним завданням автоматичного тестування. Завдяки детальному покриттю коду тестами вдається уникати серйозних проблем у продуктивному середовищі. Автоматичні тести діють як запобіжний механізм, який блокує впровадження коду з помилками, що забезпечує надійність усієї системи.

Крім основних функцій, автоматичне тестування в CI/CD сприяє більшій узгодженості командної роботи. Усі члени команди мають доступ до результатів тестів і можуть вчасно реагувати на виявлені проблеми. Інтеграція тестів із CI/CD інструментами створює єдиний робочий процес, що об'єднує розробку, тестування та розгортання.

Автоматичне тестування також дозволяє створювати стабільні релізи навіть у проєктах із високою частотою змін. Використання автоматизації зменшує людський фактор і забезпечує постійний моніторинг стану коду, що особливо важливо для Agile- та DevOps-команд. У підсумку, автоматичне тестування є невід'ємною частиною CI/CD процесів, яка забезпечує їхню ефективність і стабільність.

Автоматичні тести є ключовою частиною процесів CI/CD, забезпечуючи контроль якості коду на кожному етапі його впровадження. Робочий процес автоматичного тестування починається з внесення змін у код і завершується розгортанням стабільного програмного забезпечення в продуктивному середовищі.

Коли розробник створює або змінює код, він комітить його в репозиторій, наприклад, GitHub, GitLab або іншу систему контролю версій. Після кожного коміту CI/CD система автоматично реагує, активуючи визначені конвеєри тестування.

Система CI/CD (наприклад, Jenkins, GitLab CI/CD, CircleCI або Travis CI) запускає серію автоматичних тестів. Це можуть бути різні типи тестів, як-от:

- юніт-тести, які перевіряють окремі функції або модулі на базовому рівні;
- інтеграційні тести, що перевіряють взаємодію між різними модулями чи компонентами;

- продуктивні тести, які аналізують швидкодію системи;
- тести безпеки, спрямовані на виявлення вразливостей у системі.

Якщо всі тести проходять успішно, CI/CD система розгортає зміни в тестовому середовищі для додаткової перевірки або для роботи QA-команди. На цьому етапі також можуть запускатися додаткові автоматизовані сценарії тестування, такі як регресійні або функціональні тести.

У разі успішного проходження тестування в тестовому середовищі система автоматично або з мінімальним втручанням команди розробки деплоїть код у продуктивне середовище. Якщо ж один або кілька тестів завершуються невдачею, процес розгортання блокується, а розробник отримує звіт із деталями про знайдені помилки. Це дозволяє оперативно виправити проблему і повторити процес.

Такий підхід до автоматичного тестування в CI/CD допомагає не лише виявляти помилки на ранніх етапах, але й забезпечувати безперервну інтеграцію та стабільну доставку програмного забезпечення. Він є невід'ємною частиною сучасного DevOps-підходу, де швидкість і якість розробки мають критичне значення.

Інструменти для автоматизації в CI/CD

Інструменти для автоматизації CI/CD відіграють важливу роль у забезпеченні безперервної інтеграції, тестування та розгортання коду. Вони автоматизують рутинні процеси, прискорюючи доставку програмного забезпечення та знижуючи ризик людських помилок.

Jenkins – один із найпопулярніших відкритих інструментів для CI/CD. Він забезпечує інтеграцію з різними мовами програмування, системами контролю версій та сторонніми плагінами. Jenkins дозволяє створювати складні конвеєри з багатьма етапами тестування, збірки та розгортання.

GitLab CI/CD – це вбудований інструмент у GitLab, який дозволяє розробникам легко налаштовувати CI/CD конвеєри. GitLab підтримує інтеграцію з репозиторіями, автоматичне тестування, побудову та деплой на будь-які

середовища. Завдяки зрозумілому YAML-файлу конфігурації, цей інструмент є ідеальним вибором для команд, які працюють безпосередньо з GitLab.

CircleCI – хмарне рішення для CI/CD, яке підтримує різні мови програмування та платформи. CircleCI вирізняється високою швидкістю виконання конвеєрів і гнучкістю у налаштуванні. Він ідеально підходить для невеликих і середніх команд, які потребують потужного інструменту для автоматизації, але не хочуть розгортати власну інфраструктуру.

Travis CI – простий у використанні сервіс для автоматизації тестування та розгортання. Travis CI інтегрується з GitHub і дозволяє швидко налаштувати автоматичні процеси. Це зручний вибір для проєктів з відкритим кодом завдяки безкоштовній підтримці для таких репозиторіїв.

Azure DevOps – потужна платформа від Microsoft, яка об'єднує всі етапи життєвого циклу розробки. Azure DevOps пропонує гнучкі інструменти для налаштування CI/CD конвеєрів, а також інтеграцію з іншими продуктами Microsoft, такими як Azure Cloud. Вона підтримує широкі можливості для автоматизації, тестування, моніторингу та управління проєктами.

Кожен із цих інструментів має свої переваги та підходить для різних потреб команд. Вибір інструменту залежить від розміру команди, складності проєкту та інфраструктури, з якою працює команда.

Приклад CI/CD конвеєра

CI/CD конвеєр автоматизує процеси інтеграції, тестування та розгортання, забезпечуючи безперервну доставку якісного коду. Ось детальний приклад етапів конвеєра:

Розробник комітить код у репозиторій (наприклад, GitHub, GitLab або Bitbucket). При цьому нові зміни автоматично відстежуються CI/CD системою.

На першому етапі автоматично запускаються юніт-тести, які перевіряють базову функціональність окремих модулів або компонентів. Це забезпечує швидке виявлення помилок у коді на ранньому етапі.

Далі система переходить до інтеграційних тестів, які перевіряють взаємодію між різними модулями або сервісами. Це гарантує, що новий код не порушує роботу залежностей або інтеграцій між компонентами.

На наступному етапі виконуються продуктивні тести, які перевіряють продуктивність, стабільність і витривалість системи під навантаженням. Наприклад, тестується час відгуку сервісів або обробка великого обсягу даних.

У разі успішного проходження всіх тестів система автоматично розгортає зміни в тестовому середовищі. Це середовище використовується для додаткової перевірки вручну, а також для кінцевих автоматичних тестів, таких як регресійні або тести безпеки.

Після підтвердження стабільності в тестовому середовищі код автоматично деплоїться в продуктивне середовище. Це дозволяє користувачам миттєво отримати доступ до нових функцій або виправлених помилок.

Цей процес може також включати додаткові етапи, такі як:

- автоматичне створення повідомлень про результати тестування;
- перевірка якості коду за допомогою статичного аналізу;
- створення резервних копій перед розгортанням;
- сповіщення команди про статус конвеєра через Slack, електронну пошту або інші інструменти.

Такий підхід дозволяє швидко доставляти якісний код, скорочуючи час між написанням змін і їхнім використанням кінцевими користувачами.

Переваги автоматичного тестування в CI/CD:

Автоматичне тестування є ключовим елементом процесу CI/CD, надаючи низку значних переваг для команд розробки:

- *швидкий зворотний зв'язок*: автоматичне тестування забезпечує розробникам миттєве повідомлення про стан змін у коді. Це дозволяє оперативно виявляти та виправляти помилки, зберігаючи високий темп розробки;

- *підтримка високої частоти випусків*: завдяки автоматизації перевірок код можна швидко доставляти у вигляді нових функцій чи оновлень, не витрачаючи час на ручне тестування. Це особливо актуально для проєктів, що працюють у рамках Agile або DevOps підходів;
- *мінімізація людських помилок*: ручне тестування часто супроводжується ризиком пропущених дефектів або помилок через людський фактор. Автоматизація дозволяє уникнути таких ситуацій, забезпечуючи повторюваність і точність процесу;
- *економія часу та ресурсів*: виконання тестів автоматично значно скорочує час перевірки коду та дозволяє командам зосередитися на створенні нових функцій. Це також знижує потребу у великих командах тестувальників;
- *покращення надійності розгортань*: інтеграція тестування в конвеєр CI/CD гарантує, що тільки якісний і перевірений код потрапляє у продуктивне середовище. Це мінімізує ризик дефектів у релізах;
- *прозорість процесу розробки*: автоматичне тестування вбудовується в CI/CD конвеєр, що дозволяє всім учасникам команди стежити за статусом тестів, розгортанням і потенційними проблемами в реальному часі;
- *масштабованість*: автоматизовані тестові системи легко адаптуються до зростання проєкту, дозволяючи ефективно обробляти збільшення кількості тестів, функціональності та інтеграцій.

Інтеграція автоматичних тестів у CI/CD значно підвищує ефективність, надійність та швидкість розробки. Це робить процес більш керованим і прозорим, що є особливо важливим для великих команд або проєктів із високими вимогами до якості та швидкості.